
Programming Paradigms

Dr. Harish D. Gadade (डॉ. हरिष डी. गडदे)
Dept. of Computer Science and Engg.
COEP Technological University, Pune

Programming Paradigms

Programming Paradigm are of two types

- Declarative Programming
 - SQL, HTML, Functional and Logical Programming
- Imperative Programming
 - Modular Programming
 - Procedural Programming
 - Structured Programming
 - Object Oriented Programming

Declarative Programming

Declarative Programming is a programming paradigm where you describe what you want to achieve, not how to do it.

In other words, you tell the computer **the goal or the result**, and the system figures out **the steps** to reach it.

◆ Characteristics

- Focus on results, not on control flow.
- No explicit loops or step-by-step instructions.

Example :

1. SQL (Structured Query Language):

You declare *what data you want*, not *how to retrieve it*.

```
SELECT name FROM students WHERE marks > 80;
```

Declarative Programming

2. HTML:

You declare *what elements should appear on a webpage*, not *how they're rendered*.

```
<h1>Welcome</h1>
```

3. Prolog (Logic Programming):

You declare *facts and rules*, and Prolog deduces answers.

```
parent(john, mary).  
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
```

4. Functional Programming (like Haskell):

```
sum = foldl (+) 0 [1..5]
```

Functional Programming

- The Functional Programming Paradigm is a declarative programming approach that views computation as the evaluation of mathematical functions and avoids changing state and mutable data.
- It focuses on what to solve rather than how to solve it.
- In functional programming (FP), programs are built using functions as the primary building blocks.
- Each function takes some input and produces an output without modifying data
- Popular Programming Languages are
 - Haskell
 - Lisp
 - Erlang
 - Scala, F#, Clojure
 - Python : Supports functional features like lambda function, map(), filters()

Functional Programming

Key Features

1. Pure Functions

- A function's output depends only on its input.
- It has **no side effects** (does not modify global variables, files, etc.).

Example:

```
def add(a, b):  
    return a + b
```

2. Immutability

- Data cannot be changed once created.
- New values are created instead of modifying existing ones.

Functional Programming

Key Features

3. First-Class and Higher-Order Functions

- Functions are treated like variables: they can be passed as arguments, returned from other functions, and stored in data structures.

Example:

```
def square(x): return x * x
def apply(func, value): return func(value)
print(apply(square, 5)) # Output: 25
```

4. Recursion Instead of Loops

- FP languages use recursion for iteration since loops often depend on mutable state.

```
factorial 0 = 1
```

```
factorial n = n * factorial (n - 1)
```

→ This is using Haskell

Imperative Programming

Imperative Programming is a programming paradigm where you tell the computer **how to do something** step-by-step.

It focuses on **describing the exact sequence of operations** that change a program's state (using variables, loops, and conditionals).

Example :

```
sum = 0
for i in range(1, 6):
    sum = sum + i
print(sum)
```

Here, you are explicitly telling the computer:

1. Start with `sum = 0`
2. Repeat for `i = 1 to 5`
3. Add `i` to `sum`
4. Print the result

Imperative Programming

◆ Features

- Focus on “how” to perform a task (step-by-step).
- Uses **variables** to store data.
- Uses **control structures** like loops (`for` , `while`) and conditionals (`if`).
- The program has a **mutable state** (data values can change over time).

◆ Common Imperative Languages

- C
- C++
- Java
- Python
- Fortran
- Pascal

Modular Programming

A **module** is a **self-contained block of code** — such as a function, class, or file — that performs a specific task and can be reused in other parts of the program.

```
// module1.c
int add(int a, int b) {
    return a + b;
}
```

```
// main.c
#include <stdio.h>
#include "module1.h"

int main() {
    printf("Sum = %d", add(3, 4));
    return 0;
}
```

Procedural Programming

- ◆ Procedural Programming is a programming paradigm based on the concept of procedures (also called routines, subroutines, or functions) — blocks of code that perform specific tasks.
- ◆ Break down a program into a series of steps or procedures that are executed in a specific order to solve a problem.
- ◆ Characteristics
 - Program is divided into functions or procedures.
 - Uses variables, loops, and conditionals to control flow.
 - Follows a top-down approach — start from the main function and divide into smaller sub-tasks.
 - Data is global or passed between functions as parameters.
 - Focuses on how to do a task (sequence of instructions).

Procedural Programming

◆ Examples of Procedural Languages

- C
- Pascal
- FORTRAN
- BASIC
- Python (can be used procedurally)
- JavaScript (can also be used procedurally)

◆ Structure of a Procedural Program

1. Variable declarations
2. Functions (procedures)
3. Main program flow

```
#include <stdio.h>

void greet() {
    printf("Hello, COEP Technological University!\n");
}

int add(int a, int b) {
    return a + b;
}

int main() {
    greet();
    int result = add(5, 10);
    printf("Sum = %d\n", result);
    return 0;
}
```

Structured Programming

- Structured programming languages use clear, logical control structures—like sequence, selection, and iteration—to make code easier to read, debug, and maintain. Popular examples include C, C++, Java, and Python.
- Structured programming is a paradigm that emphasizes organizing code into well-defined blocks using:
 - **Sequence:** Executing instructions one after another.
 - **Selection:** Making decisions using if, else, or switch statements.
 - **Iteration:** Repeating actions using loops like for, while, or do-while
- This approach avoids the use of GOTO statements

Structured Programming

Advantages

- Easier to read and understand
- User Friendly
- Easier to Maintain
- Mainly problem based instead of being machine based
- Development is easier as it requires less effort and time
- Easier to Debug
- Machine-Independent, mostly.

Disadvantages

- Since it is Machine-Independent, So it takes time to convert into machine code.
- The converted machine code is not the same as for assembly language.

Object Oriented Programming

- ◆ Object-Oriented Programming (OOP) is a programming paradigm based on the concept of “objects”, which can contain **data** (in the form of fields, often called *attributes*) and **code** (in the form of *methods*).
- ◆ The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.
- ◆ **OOPS Concepts:**
 - Class
 - Object
 - Encapsulation
 - Abstraction
 - Inheritance
 - Polymorphism

Object Oriented Programming

1. Class

- A class is a user-defined data type. It consists of data members and member functions, which can be accessed and used by creating an instance of that class.
- It represents the set of properties or methods that are common to all objects of one type.

```
class Student {  
    public:  
        string name;  
        int age;  
        void display() {  
            cout << name << " is " << age << " years old.";  
        }  
};
```


Object Oriented Programming

2. Object

- It is a basic unit of Object-Oriented Programming and represents the real-life entities.
- An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.
- For example “Dog” is a real-life Object, which has some characteristics like Breed, Age, Color, etc.

```
Student s1;  
s1.name = "Harish";  
s1.age = 21;  
s1.display();
```

Object Oriented Programming

3. Encapsulation

- It is the mechanism that binds together code and the data it manipulates.
- In Encapsulation, the variables or data of a class are hidden from any other class and can be accessed only through any member function of their class in which they are declared.
- As in encapsulation, the data in a class is hidden from other classes, so it is also known as data-hiding.

```
class Account {  
    private:  
        double balance;  
    public:  
        void deposit(double amt) { balance += amt; }  
        double getBalance() { return balance; }  
};
```

Object Oriented Programming

4. Abstraction

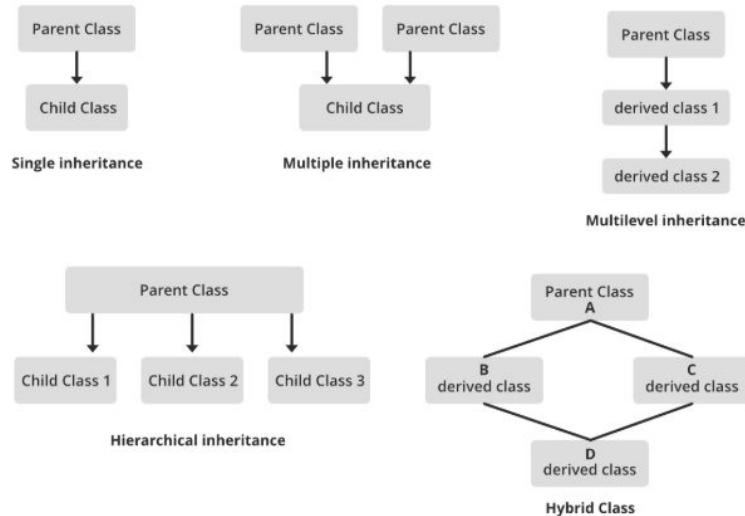
- Data abstraction is one of the most essential and important features of object-oriented programming.
- Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.
- Hiding complex implementation and showing only necessary details.
- Consider a real-life example of a man driving a car.

```
class Car {  
    public:  
        void start() { cout << "Car started"; } // Simple interface  
};
```

Object Oriented Programming

5. Inheritance

- Deriving a new class from an existing one to reuse code.
- The capability of a class to derive properties and characteristics from another class is called Inheritance.



```
class Vehicle {  
    public:  
        void start() { cout << "Starting vehicle"; }  
};  
  
class Car : public Vehicle {  
    public:  
        void honk() { cout << "Beep!"; }  
};
```

Object Oriented Programming

6. Polymorphism

- The word polymorphism means having many forms.
- Ability to perform a single action in different forms.
- Achieved through function overloading
- A person at the same time can have different characteristics. Like a man at the same time is a father, a husband, an employee. So the same person possesses different behavior in different situations. This is called polymorphism.

```
int add(int a, int b) {  
    return a + b;  
}
```

```
// Same function with different arguments  
int add(int a, int b, int c) {  
    return a + b + c;  
}
```